

Cube Maintenance

当数据源的数据发生变化时，需要对数据方体（采用各种压缩方法缩减后的数据方体）中的数据进行相应的维护。

重新计算：简单方法，计算量太大

增量维护：在已有的数据方体基础上进行调整。

Cube Maintenance

- 对于不同的聚集函数，增量维护的方法和代价不一样
 - 分布型聚集函数：具有可以累计的特性，非常适合于增量维护。这类函数称为可自维护型 (**self-maintainable**) 聚集函数。
 - 代数型聚集函数：可以转化为分布函数，也是可自维护型函数。
 - 整体型聚集函数：在计算的时候需要所有的数据，很难进行增量维护，被称为不可自维护型

Cube Maintenance

- 大多数增量维护的算法都是针对可自维护型函数。
- 对不可自维护型函数，通常采用近似的方法，维护一个统计信息，当数据源发生改变的时候，对统计信息进行更新，然后根据更新后的统计信息估计查询结果。

Cube Maintenance

- 设计增量维护算法的困难主要在于：
 - 要同时支持三种类型的聚集函数。
 - 维护算法的效率必须高于重新计算的效率，
 - 必须能够对增加的数据进行批量更新。
 - 设计好的批量更新算法是至关重要的。

Cube Maintenance

- 介绍我们自己的一项工作
- 针对Quotient Cube进行维护
 - Incremental maintenance of quotient cube for median. KDD 2004: 226-235

Condensed 和 Quotient比较

a1b1c1

a1b1c2

a2b2c1

- Quotient cube: 分为5个等价类
- Condensed Cube
 - a1b1c1 是BC AC ABC上的单元组
 - a1b1c2 是BC AC ABC上的单元组
 - a2b2c1 是A B AB AC BC ABC上的单元组

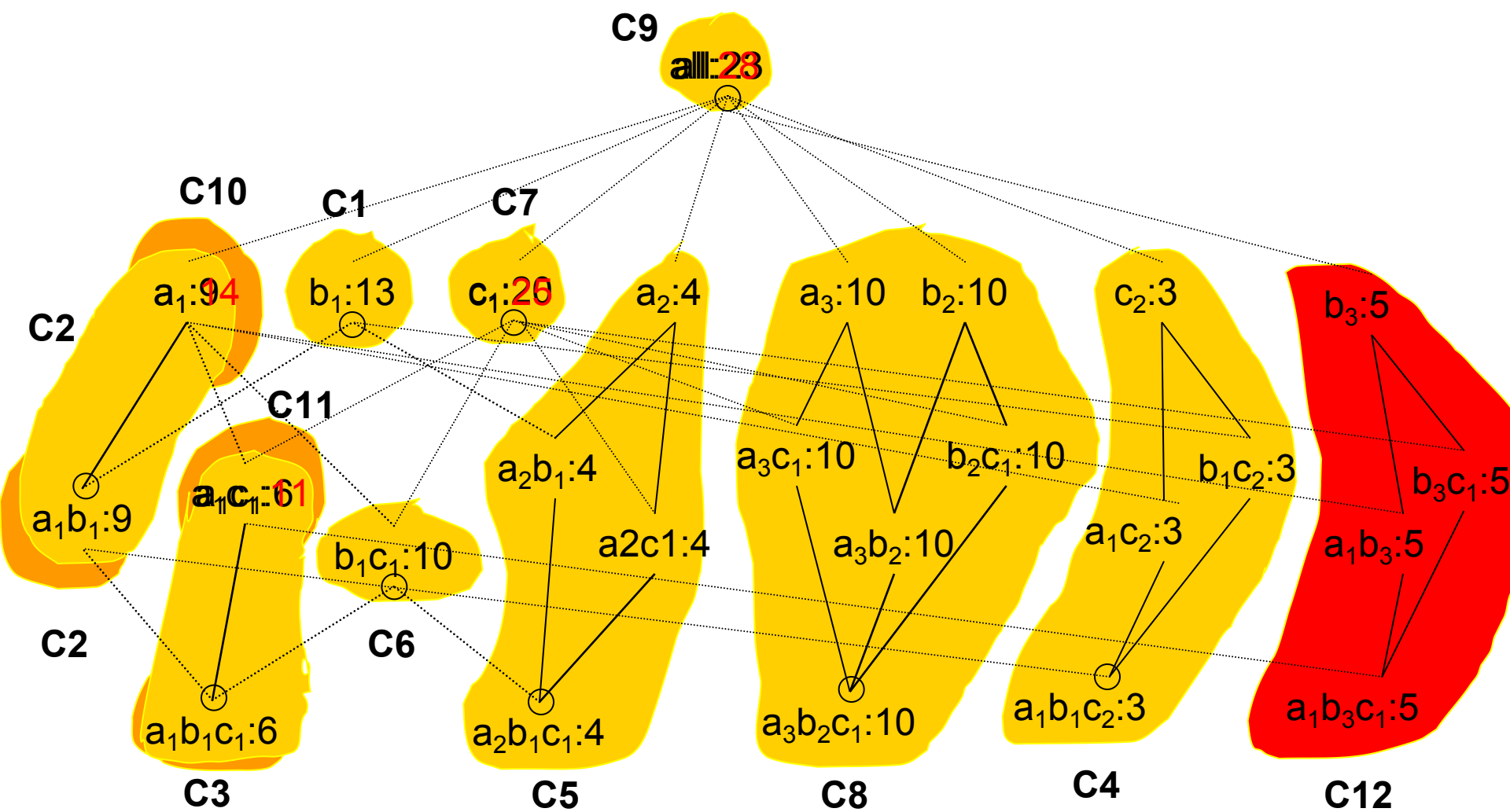
-
- 每个等价类有上届和下届，上届是唯一的。
 - 每个等价类本身是一个格结构
 - 上届和所对应的基元组**ID**集合之间是最大匹配。
 - 计算**Quotient Cube**的过程就是求每个等价类的下界和上届的过程。
 - 运用**BUC**算法求上届的过程体现**close**原理

Quotient Cube的维护

挑战:

- Quotient Cube的等价类会变。
 - 例如，当插入一条新的元组时，原来的等价类可能需要分解。
 - 而当删除一条元组时，原来的等价类可能需要合并
- 当聚集函数是**Median**等综合聚集函数时，维护更难。
 - 因为计算新的聚集值需要所有的历史数据，无法实现增量维护。
时间和空间开销太大

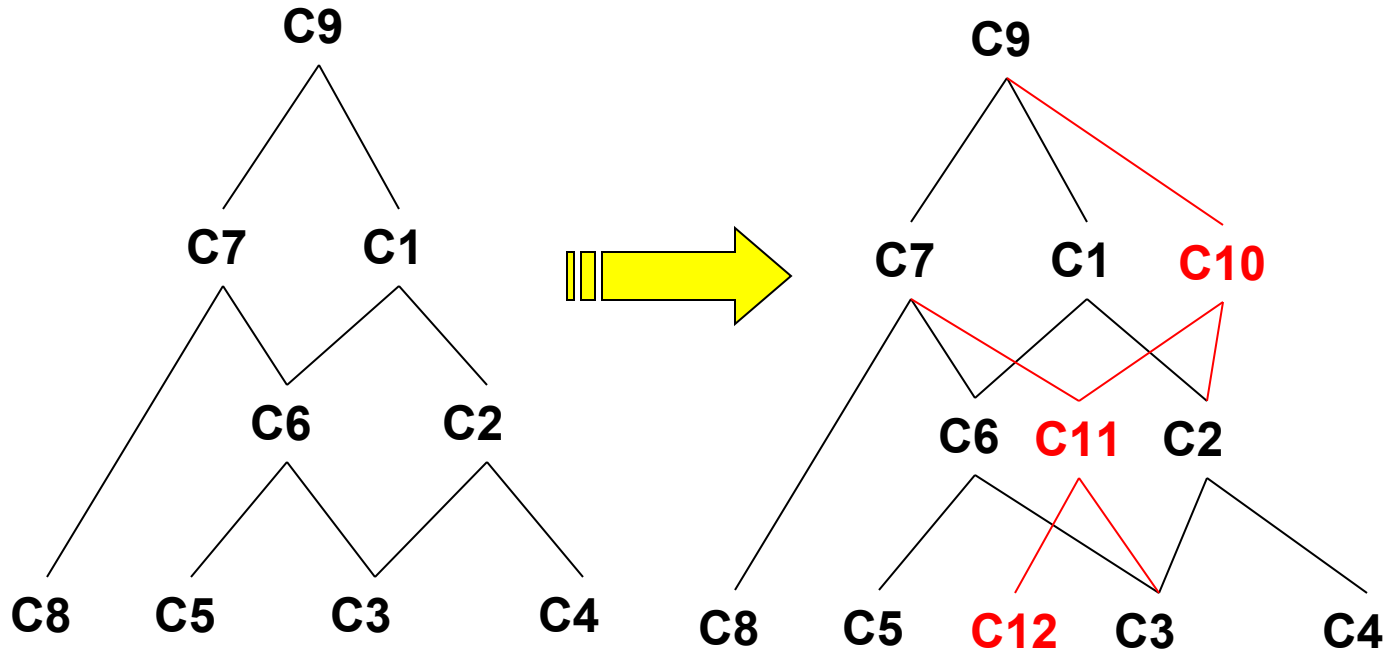
一个例子



维护策略

- 新插入一条元组可以对一个已有的等价类作如下改变
 - 使某个等价类的聚集值发生改变 (Value Modified Class)
 - 使某个等价类分裂成两个 (New Class and its Generator)
- 还有一些等价类不做任何改变 (Dump Class)

维护策略(Cont.)



- C7,C9 是value modified class
- C10是新 class, C2是generator
- C11是新 class, C3是generator
- C12是新 class, no generator (special case, introducing virtual class)
- 其他是dump class

维护策略(Cont.)

- 找出各种类型等价类的特征，针对不同类型的等价类对它们分别进行维护

维护策略(Cont.)

- 观察发现，当一个新的元组和一个旧的等价类的上届匹配的时候，新元组的插入将会引起等价类聚集值的改变
- 由于等价类的上届是最综合的数据元素，只要元组与它匹配，则与该等价类中的每一个数据元素都匹配。因此，在这种情况下，旧的等价类不需要分裂。

维护策略(Cont.)

■ 属性1 [修改类]

如果一个类 C_m 满足条件: $C_m.\text{upp} \subseteq t.\text{cell}$

则该类是一个需要修改的类, $C_m.v = C_m.v + t.v$ 。

- t 的维值集合 $t.\text{cell}$ 如果包含类 C_m 的上届 $C_m.\text{upp}$, 则意味着元组 t 对类 C_m 的聚集值有贡献, 因此需要对类 C_m 的聚集值进行更新。
- 比如, 由于 $c1 \subseteq a1b3c1$, 等价类 $C7$ 的聚集值从20改为25。

维护策略(Cont.)

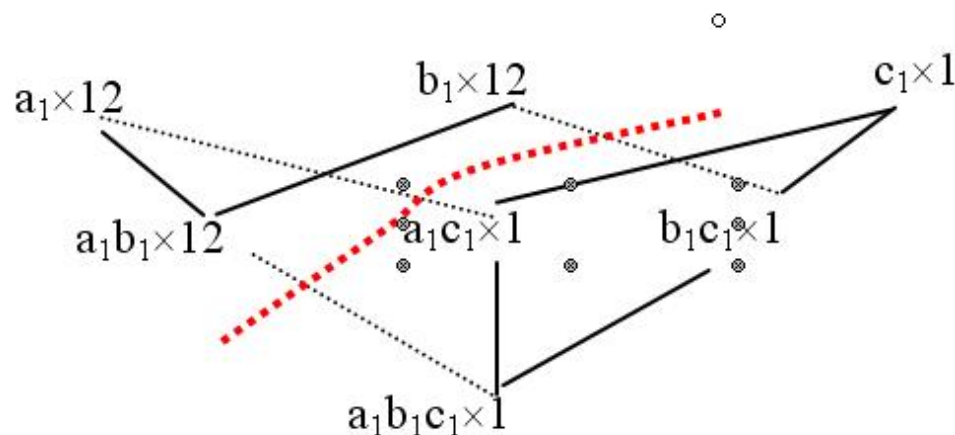
- **属性2 [新类的特征]**

新类 C_n 具有特征 $C_n.\text{upp} = C_g.\text{upp} \cap t.\text{cell}$,
其中 C_g 是一个旧类。 C_g 称为 C_n 的生成者。

- 元组 t 的维值集合 $t.\text{cell}$ 和某个旧类的上届相交表示元组 t 对类 C_g 中的部分元素的聚集值作出了贡献。由于此时旧类 C_g 的所有元素所包含的底层元组不一样了，所以此时旧类 C_g 必须拆分成两个类。

维护策略(Cont.)

- 一个极端的例子：假设基表R中原来只有一个元组,
 $a_1 \ b_1 \ c_1 \ 10$
- 旧的quotient数据方体只有一个类
 - 上届是: a_1, b_1, c_1
 - 下界是: $a_1 b_1 c_1$
- 往R里加一个新元组
 $a_1 \ b_1 \ c_2 \ 5$



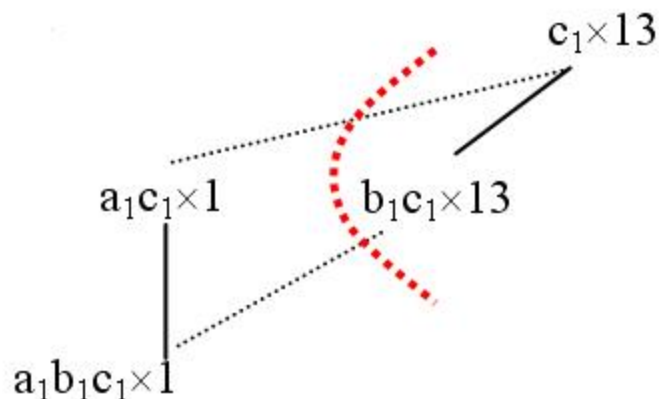
维护策略(Cont.)

- C1分成两个类:
 - C2: 下界是a1, b1 上届是a1b1
 - C1': 下界是c1 上届是a1b1c1
- 新类C2是从旧类C1拆分出来的。C1被称作C2的生成者。请注意，拆分之后旧类C1 的下届从{a1, b1, c1}变为{c1}。

维护策略(Cont.)

- 再次加入元组:

a2 b1 c1 5



- 维护的重要任务就是：从旧类中找出那些需要进行拆分的类（生成者），然后由它们生成新类。

维护策略(Cont.)

■ 属性7.3 [生成者的特征]

如果不存在某个旧类 $Z \in \Phi$ 使得 $X.\text{upp} = Z.\text{upp}$, 则 $C_g = \text{GLB}\{Y \in \Phi \mid X.\text{upp} = Y.\text{upp} \cap t.\text{cell}\}$ 是新类 $X \in \Phi'$ 的生成者。

可能有多个旧类能够和 $t.\text{cell}$ 相交并产生相同的交集。这时, 类格的GLB (Greatest Lower Bound) 是生成者。

维护策略(Cont.)

- 考虑相继插入如下元组的情况
 - a1b1c1
 - a1b1c2
 - a1b2c2

维护算法

- 采用了两种技术
 - Addset数据结构
 - Sliding Window技术

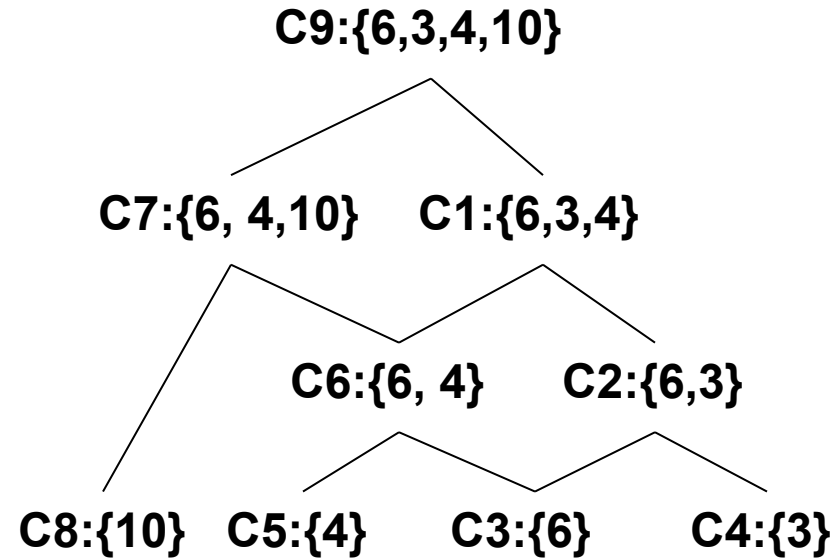
Addset 数据结构

- Naive Method to maintain median
 - Store a set of measure (**Measure Set**) for each cell
 - Update the measure set and re-compute the aggregation Median when new tuples are inserted.

Problem: Expensive for the large number of cells and tuples

Addset data structure: Motivation (cont.)

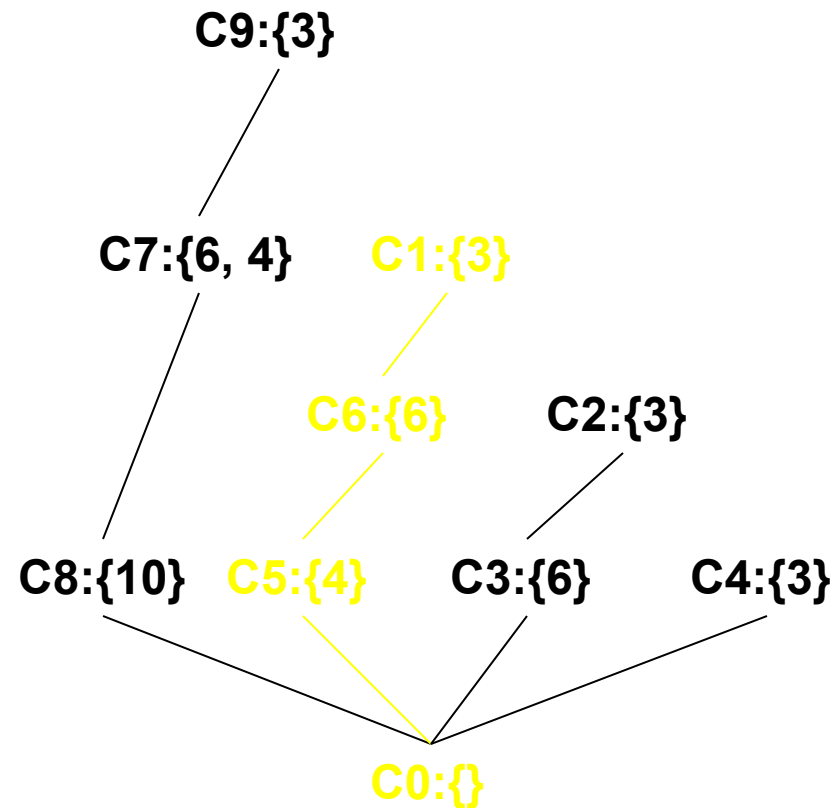
- Quotient cube helps to reduce this storage requirement
 - Store only one **measure set** for each equivalence class



Problem: Redundancy! For example: **6,4,10** in $C7$ and $C9$

Naïve Addset data structure

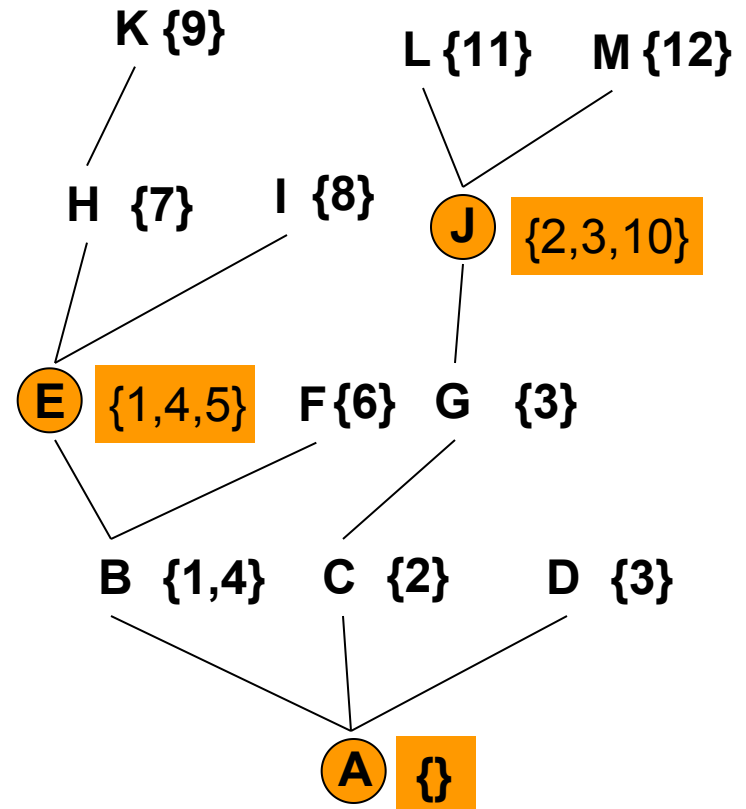
- Store the measure set difference between new class and its generator (**Addset**)
- Space saving: about **2** times (from 18 to 10)
- Family tree
- Combine the addset along its family linkage path, we get the measure set, for example:
 $MS(C1) = \{3\} \cup \{6\} \cup \{4\} = \{3, 6, 4\}$



Problem: Measure set computation cost **increases** with the length of family path

Dynamical Materialization of Addset

- In order to get some tradeoff between space and time, dynamically **materialize** some class in the process of maintenance
 - Pseudo class: store addset
 - Materialized class: store actual measure set
- To obtain the actual measure set of a pseudo class
 - only need to trace to **its nearest materialized ancestor class** instead of tree root.



Dynamical Materialization of Addset(Cont.)

- Which classes should be materialized?
 - Similar to materialized view selection. If there is sufficient space, we can materialize more to save maintenance time; Otherwise, keep more classes to be pseudo to save storage space.
 - In our case, the **distance** threshold is used to control the materialization of pseudo class, when the Distance between a pseudo class and its nearest exceeds the given threshold, it will materialized.

Slide Window

- Motivation

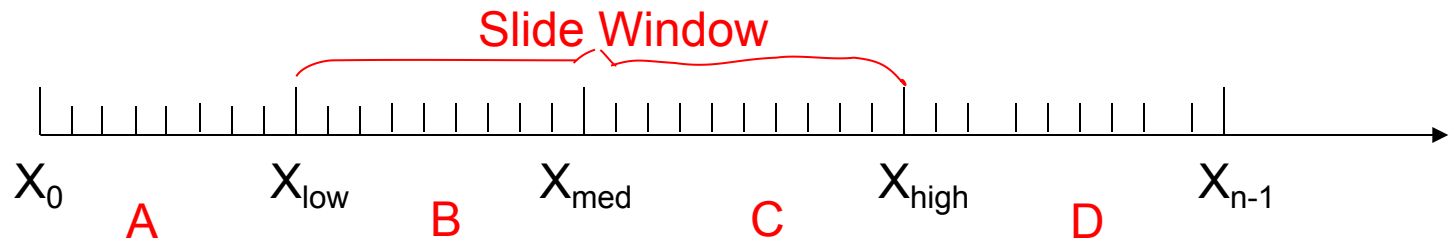
- The addset reduces the storage requirement, however, computation of updated MEDIAN is still expensive.
- Moreover, extra processing is required to obtain the measure sets for pseudo class.

- Important observation

- the number of elements that are larger and smaller than the median of n measures is the same
- By keep k measures in the sliding window, the median will still lie in the window even with k insertions.

Slide Window (Cont.)

- Given a materialized class C_m with a sorted measure set $S=\{X_0, X_1, \dots, X_{n-1}\}$, the median of one of its descendant pseudo class C_d can be computed as follows:



- A:** X_{med} move $\frac{1}{2}$ position to left
- B:** X_{med} move $\frac{1}{2}$ position to left, X_{low} move 1 position to the right
- C:** X_{med} move $\frac{1}{2}$ position to right, X_{high} move 1 position to the left
- D:** X_{med} move $\frac{1}{2}$ position to right

Slide Window (Cont.)

- Restriction: Distance between C_m and C_m can not exceed the size of the slide window.
- Solution: let the distance threshold smaller than the size of the sliding window.
- Interestingly, the size of the sliding window can be a parameter to balance the space-time tradeoff.
 - Equal to 1: all class are materialized, naïve quotient cube
 - Equal or larger than the total number of tuples, all class are pseudo class, highest space reduction, but need to compute and sort all measures of a class when computing median.

Maintenance Algorithms

- Single Tuple Maintenance
 - Update a median quotient cube **tuple by tuple**
 - Not so efficient
- Batch Maintenance
 - Update a median quotient cube in batch
 - Very efficient

Single Maintenance Algorithm

- Divide all class into buckets $B[0], B[1], \dots$
 - Bucket $B[i]$ contains all classes whose upper bound cardinality is i .
- Iterate through the classes in each bucket in the order $B[0], B[1], \dots$
 - Applying the proposition 1-3, modify the aggregations, split the classes,...
 - Materialize those classes whose distance exceeds threshold (addset structure)
 - Computer new median for all new and modified classes in **depth-first** order (sliding window)

Batch Maintenance Algorithm

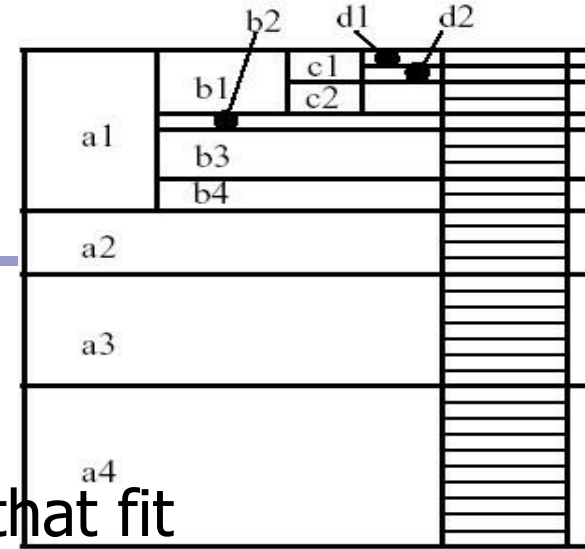
- Inspired by BUC algorithm
- Novelty of our algorithm
 - “Synchronize” partitioning on both existing classes and the new set of tuples
 - Enhances efficiency 1: by grouping thples that share similar dimensional values together, the search for affected classes needs only to be done once
 - Enhances efficiency 2: as the partitioning of classes done in synchronization with the tuple partitioning, the number of classes that are being checked is substantially reduced.

Batch Maintenance Algorithm (Cont.)

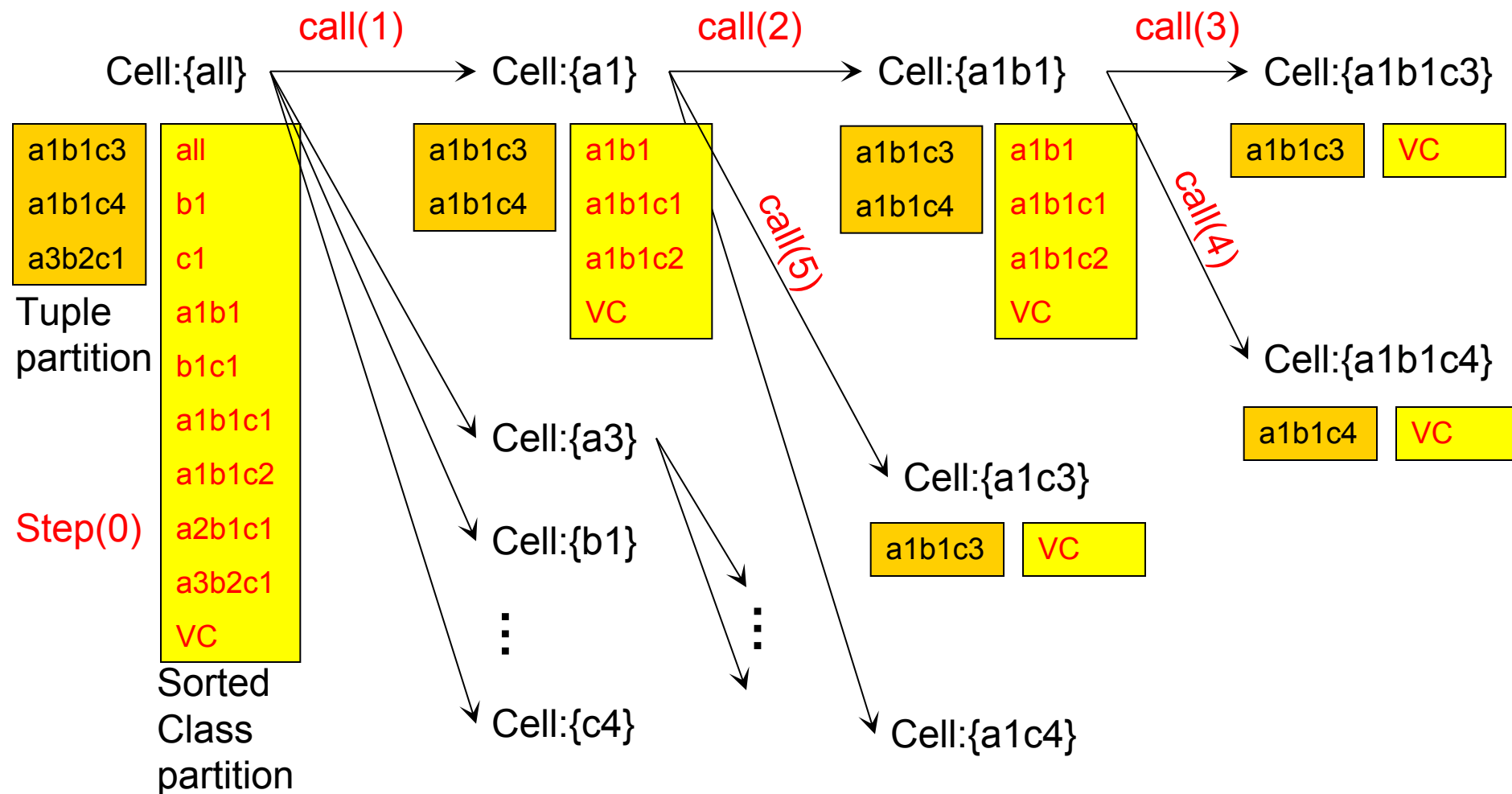
- Recursively partitioning on tuple and class in a depth-first order
- Check and update classes similar to previous Single maintenance algorithm
 - **Difference:** the cell (not the whole tuple) is used as a representative to compare against the class

BUC: Partitioning

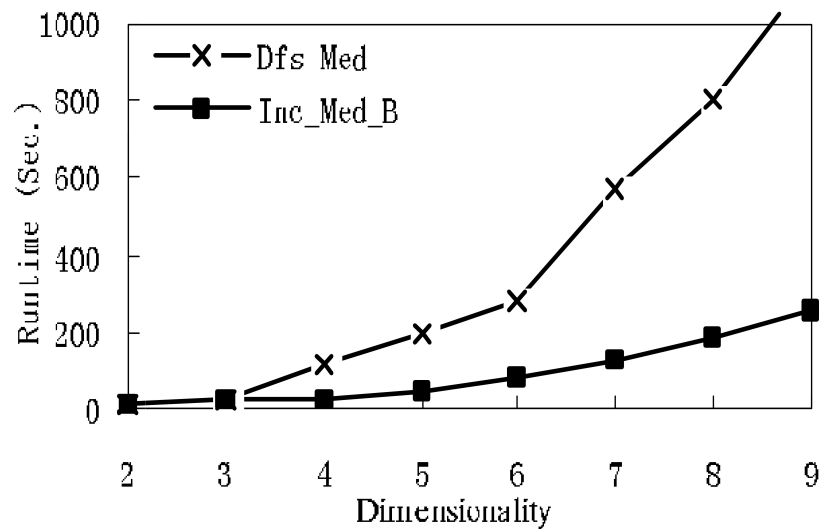
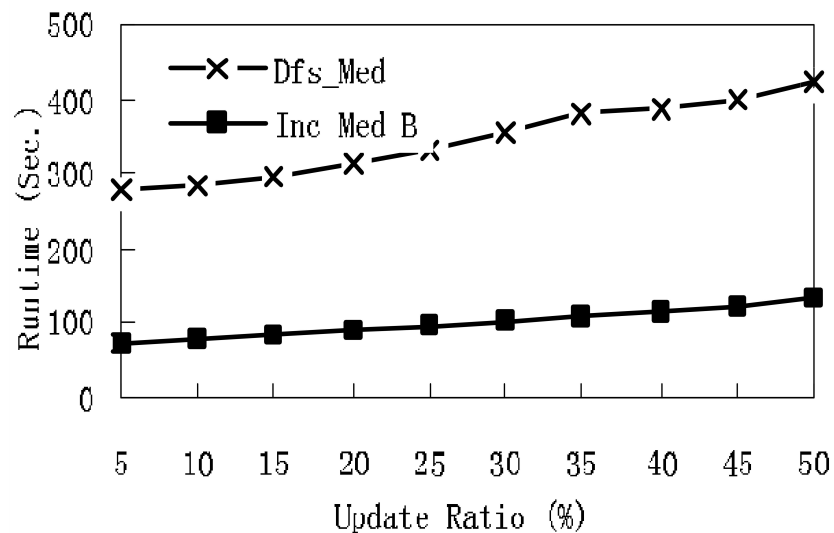
- Usually, entire data set can't fit in main memory
- Sort *distinct* values, partition into blocks that fit
- Continue processing
- Optimizations
 - Partitioning
 - External Sorting, Hashing, Counting Sort
 - Ordering dimensions to encourage pruning
 - Cardinality, Skew, Correlation
 - Collapsing duplicates
 - Can't do holistic aggregates anymore!



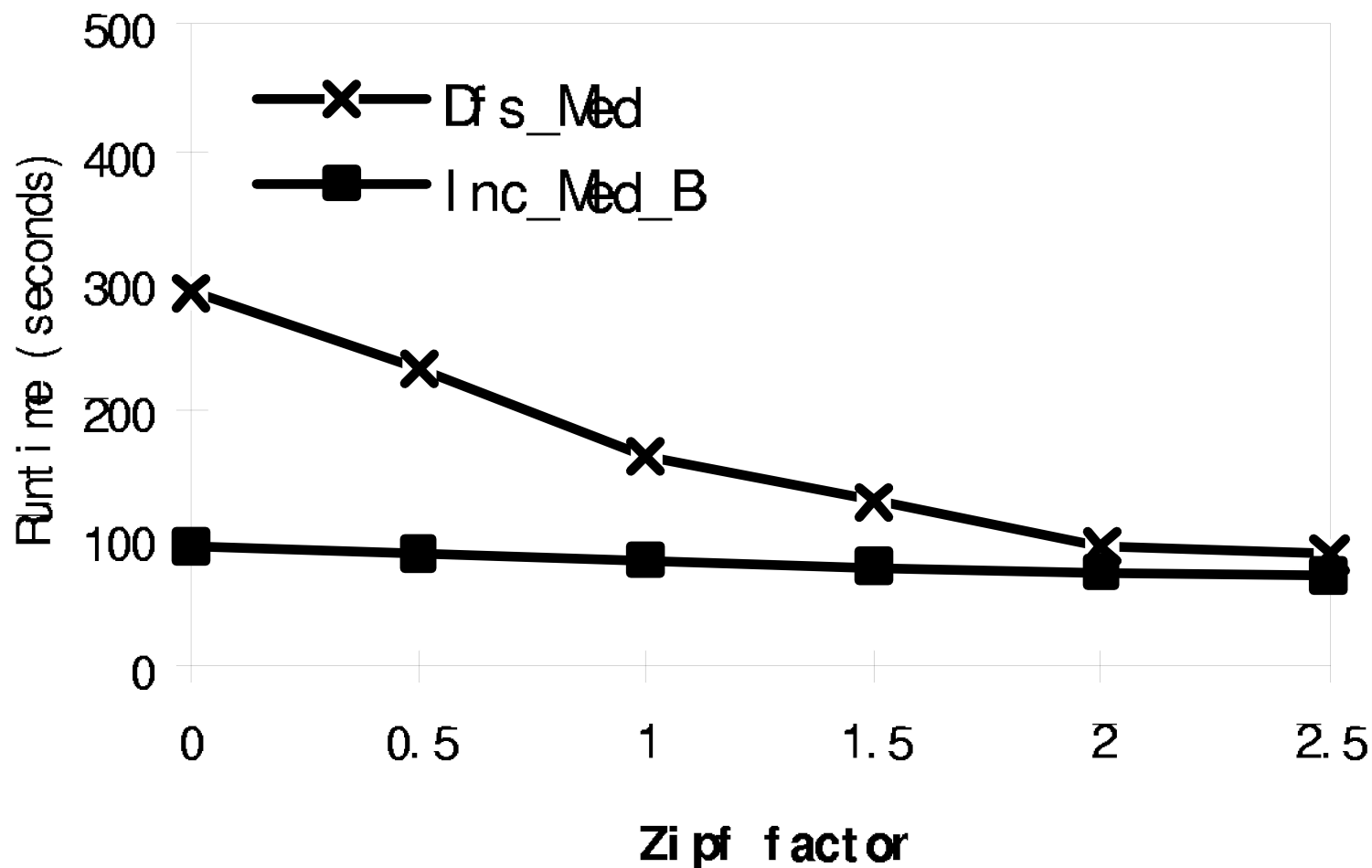
Batch Maintenance Example



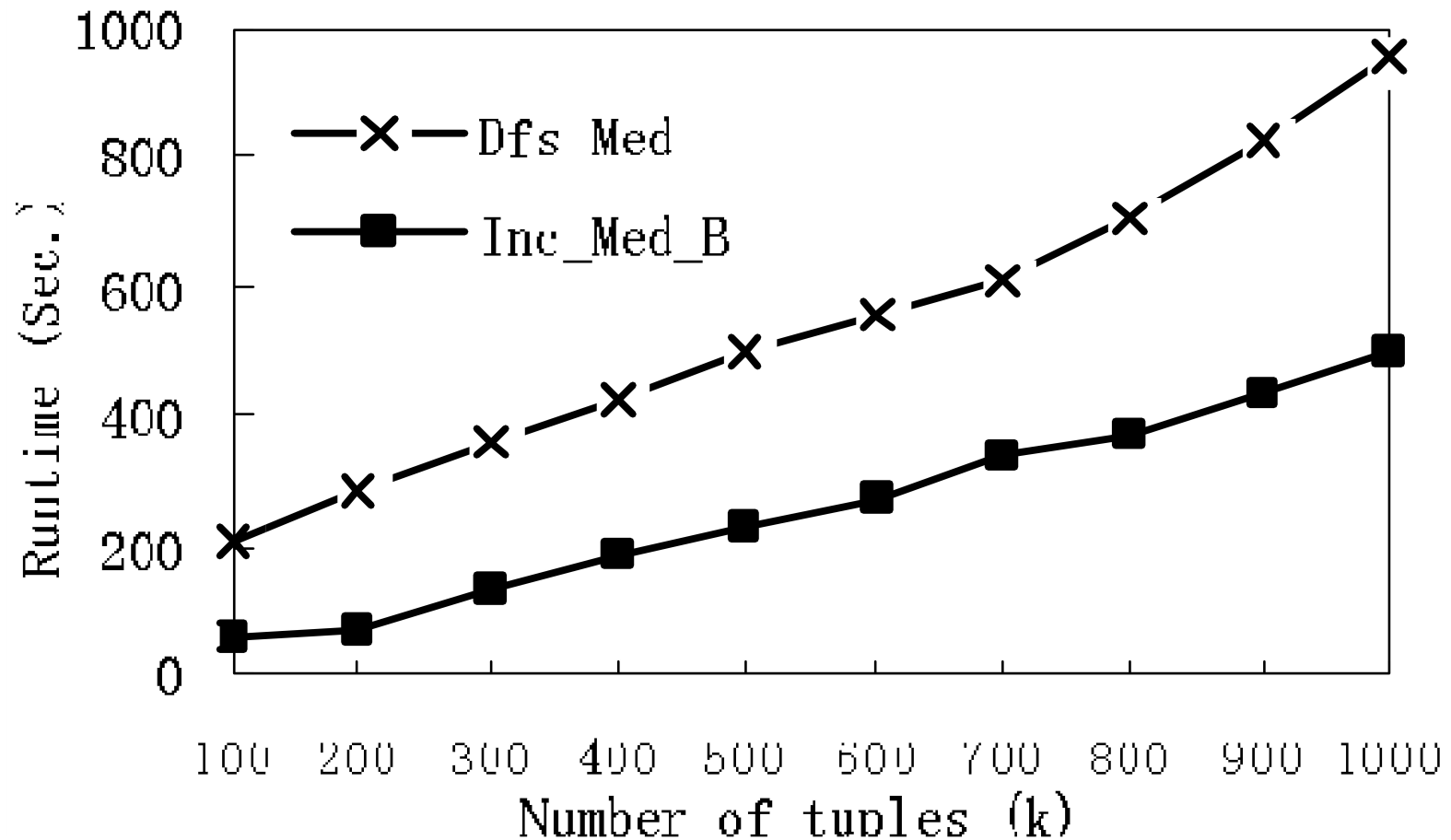
Maintenance Efficiency



Impact of data skew



Scalability with the number of tuples



Effectiveness of Addset and Sliding Window

